1. `ConfigureWebHostDefaults()`
2. `ConfigureHostConfiguration()`
3. `ConfigureAppConfiguration()`
4. `ConfigureLogging()`
5. `ConfigureServices()`
6. `Startup.ConfigureServices()`
7. `Startup.Configure()`

For the most part, this ordering detail shouldn't matter, but it still adds apparent complexity for those who are new to ASP.NET Core.

Point 5 in the list of challenges relates to the `Startup` class and the default convention/ pattern-based approach. Users coming to ASP.NET Core for the first time will likely be familiar with interfaces and base classes, but they may not have experienced the reflection-based approach.

Using conventions instead of an explicit interface adds flexibility but can make things harder for discoverability. There are also various caveats and edge cases to consider. For example, you can inject only `IWebHostEnvironment` and `IConfiguration` into the `Startup` constructor; you can't inject anything into the `ConfigureServices()` method, but you can inject any registered service into `Configure()`. These are implied rules that you discover primarily by breaking them and then having your app shout at you!

TIP The pattern-based approach allows for a lot more than DI into `Configure`. You can also create environment-specific methods, such as `Configure-DevelopmentServices` or